



Bruna Filipa Lima **Frogi**
Magarreiro
Gonçalo Conceição
da Silva Soares

Jogo desenvolvido em JavaFX no âmbito da
cadeira de Programação Orientada a Objetos

Junho 2026

Índice

1. Apresentação do Projeto	3
2. Modelação do Domínio	4
3. Diagrama de Classes UML	5
4. Arquitetura da Aplicação	7
5. Funcionalidades Implementadas	7
6. Aplicação dos Conceitos de POO	13
7. Tratamento de Exceções	14
8. Testes Unitários	15
9. Dificuldades Encontradas	16
10. Conclusão	17

1. Apresentação do Projeto

O Frogi é um jogo desenvolvido em Java utilizando a biblioteca gráfica JavaFX. O objetivo principal é guiar um sapo por um mapa dividido em secções de relva e rio, enfrentando predadores (raposas) e evitando cair à água, para conseguir alcançar a princesa localizada no nível final (nível 3).

Regras principais:

- O sapo deve alcançar a princesa, atravessando o rio através dos nenúfares e fugindo das raposas.
- Não é possível sair dos limites do mapa.
- Se o sapo ficar na mesma célula que uma raposa, perde uma vida.
- Se cair na água (fora de um nenúfar), perde uma vida.
- Se estiver em cima de um nenúfar e tentar sair do mapa, perde uma vida.
- O sapo aumenta de tamanho ao comer grilos.
- PowerUp de Salto: Ao passar por cima deste item, o sapo avança automaticamente 3 passos para a direita.
- PowerUp de Vida Extra: Ao recolher este item, o jogador ganha uma vida adicional.
- O jogo é composto por 3 níveis.
- Para completar os Níveis 1 e 2, o sapo deve alcançar a placa que diz "Prox Nvl".
- Para vencer o jogo, o jogador deve completar os três níveis e chegar até à princesa.
- O jogo termina em derrota se o jogador perder todas as suas vidas.

Interação com o jogador:

- Movimentação: O utilizador controla o sapo utilizando as setas (cima, baixo, direita, esquerda) ou as teclas WASD;
- Pausar o jogo: O utilizador pode pausar o jogo a qualquer momento premindo a tecla ESC, que irá abrir um menu de opções que incluem retomar o jogo e sair para o menu;
- Menu Inicial: O utilizador usa o rato para navegar pelas opções do menu: Novo Jogo, Opções, Pontuações e o botão de ajuda (?).
- Página de Como Jogar (botão ?): Explica o funcionamento do jogo e permite voltar ao menu clicando na seta de voltar.
- Página de Opções: Contém um painel de volume para a música e botões para alternar o jogo entre o modo Janela e Fullscreen. Permite voltar atrás clicando na seta de voltar.
- Página de Pontuações (Leaderboard): Exibe uma tabela com os 10 jogadores que obtiveram as melhores pontuações. Também tem um botão para voltar.
- Página de Novo Jogo: Onde o utilizador digita o seu nome e clica em "Jogar" para iniciar a partida, ou em "Cancelar" para regressar ao menu.

2. Modelação do Domínio

As principais entidades identificadas e a forma como se relacionam entre si são:

- **Partida** - Representa uma sessão individual do jogo. É responsável por gerir níveis, controlar as vidas, os grilos consumidos, controlar o progresso e verificar as condições de vitória e derrota.
- **Jogador** - Representa o utilizador do jogo. É responsável por armazenar o nome do jogador.
- **Sapo** - A entidade Sapo representa a personagem controlada pelo jogador. É responsável pela movimentação no mapa, evolução (determinado pelo número de grilos consumidos) e gestão do estado de vida/morte
- **Fase do Sapo** - A fase do sapo representa o estado evolutivo da personagem, sendo determinada pelo número de grilos consumidos e implementada como uma enumeração.
- **Nível** - O nível representa uma etapa do jogo e é responsável por definir a dificuldade e associar um mapa.
- **Mapa** - A entidade Mapa representa o ambiente do nível. Contém a estrutura espacial onde decorre a ação, incluindo as células que são rio e as que são relva, e todos os elementos posicionados, incluindo grilos, power-ups, a princesa, o sapo e os predadores.
- **EntidadeJogo** - A entidade EntidadeJogo é responsável por armazenar a posição das entidades e permite reutilização de código.
- **Grilo** - A entidade Grilo representa os objetos colecionáveis do jogo. Quando apanhados, aumentam o número de grilos consumidos pelo sapo, contribuindo para a evolução do sapo e para a pontuação final.
- **Predador** - A entidade Predador representa o inimigo do sapo que o mata quando o alcança.
- **Princesa** - A entidade Princesa representa o objetivo final do jogo. Apenas se torna acessível no nível final. Quando o sapo interage com a princesa, o jogo termina com vitória.
- **PowerUp** - A entidade PowerUp representa objetos especiais que aparecem no mapa e concedem vantagens ao jogador. Estes objetos possuem um comportamento comum, mas podem ter efeitos distintos.
 - **VidaExtra** – concede uma vida extra ao jogador.
 - **Salto** – permite ao jogador avançar 3 passos para a direita, saltando o rio.
- **ResultadoPartida** - A entidade ResultadoPartida representa o resultado de uma partida individual. É responsável por armazenar e calcular a pontuação final do jogador, com base em critérios definidos. Inclui:
 - jogador associado
 - número de grilos apanhados
 - tempo decorrido
 - pontuação total

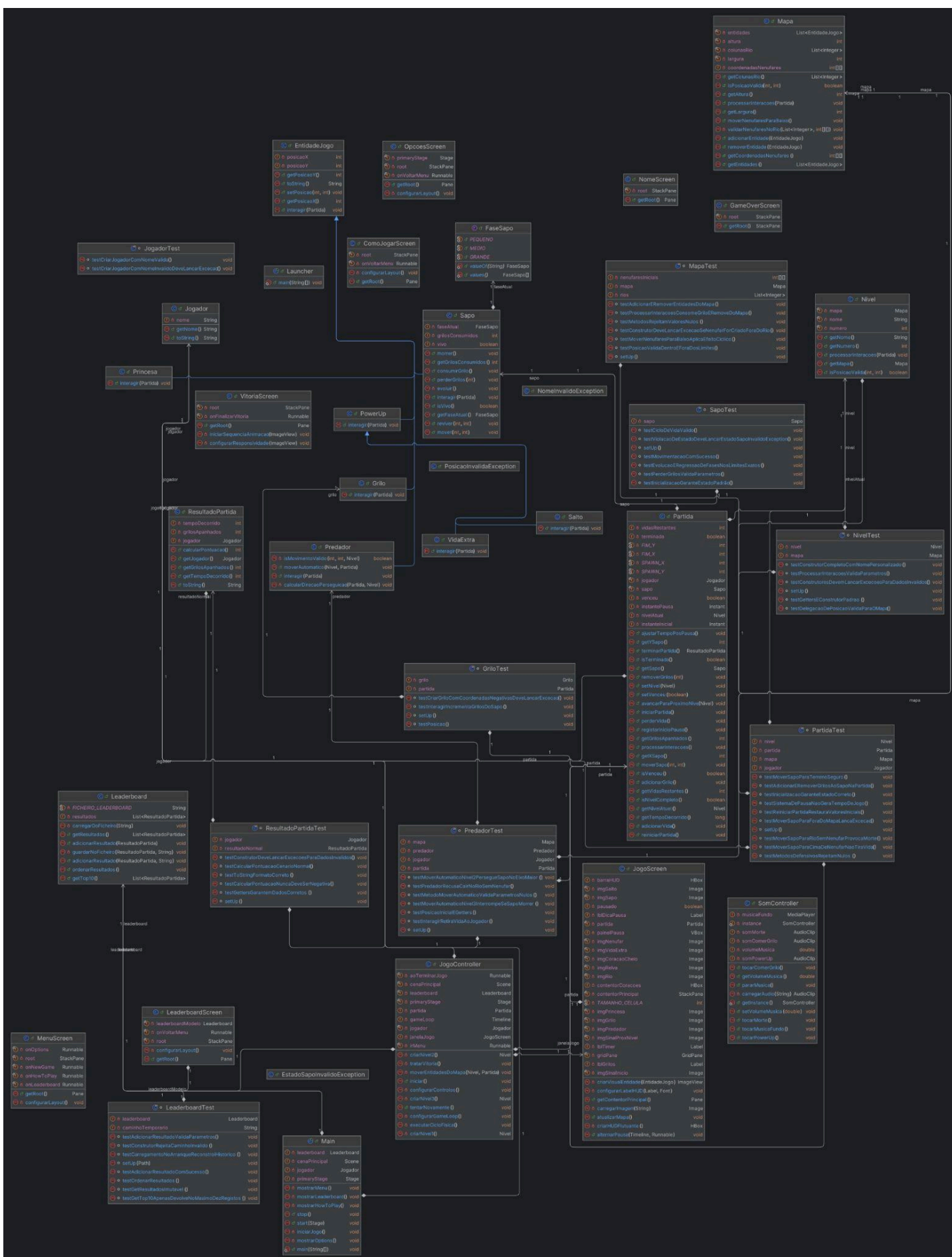
A pontuação é calculada considerando simultaneamente o desempenho (grilos recolhidos) e a eficiência (tempo).

- **Leaderboard** - A entidade Leaderboard é responsável por armazenar e organizar os resultados das várias partidas. Gere um ficheiro com os resultados que mantém

ordenados de acordo com a pontuação obtida. Permite registrar novos resultados, ordenar os resultados por pontuação e apresentar os melhores jogadores (top 10)

3. Diagrama de Classes UML

Devido ao tamanho, o diagrama UML encontra-se disponível também no zip do projeto, na mesma pasta que o relatório (/docs).



4. Arquitetura da Aplicação

O projeto foi dividido em três partes independentes para garantir uma boa organização do código:

Modelo de Domínio: Este ficou organizado no diretório `org.frogi.model`, onde foram criados mais dois diretórios para melhor organização, `org.frogi.model.entidades` e `org.frogi.model.exceptions`. Aqui está centralizada a lógica da aplicação, o que inclui todas as classes relativas a entidades, lógica do jogo e as exceções personalizadas. Esta parte não mexe com imagens nem com janelas (JavaFX), servindo apenas para processar as regras do jogo.

Controlo da Aplicação: Fica no diretório `org.frogi.controller`, onde está a classe `JogoController` e `SomController`. O `JogoController` funciona como um intermediário. Ele deteta quando o jogador carrega numa tecla, avisa o Modelo para atualizar a posição do sapo e, logo a seguir, diz à Interface Gráfica para redesenhar o ecrã com as novidades. Também controla o Game Loop (o temporizador) que faz as raposas e os nenúfares moverem-se sozinhos. O `SomController` controla todo o som do jogo, incluindo música de fundo e efeitos sonoros.

Interface Gráfica (JavaFX): Esta parte ficou guardada no diretório `org.frogi.view`. É responsável por tudo o que o jogador vê no ecrã, como o menu principal, o tabuleiro do jogo, o ecrã de game over, a tabela de pontuações e os restantes. Também inclui a classe `EstiloBotao`, que foi criada para dar o mesmo aspeto e comportamento visual a todos os botões do jogo.

5. Funcionalidades Implementadas

As principais funcionalidades desenvolvidas e disponibilizadas na aplicação são as seguintes:

- **Navegação no Menu Inicial:** Disponibilização de um ecrã principal que permite ao utilizador aceder a todas as secções do jogo: iniciar uma nova partida, consultar as instruções, configurar as opções de sistema e visualizar as classificações.
- **Registo de Utilizador:** Mecanismo que solicita e regista o nome do jogador antes do início de cada partida, com a opção de cancelar a ação e regressar ao menu.
- **Controlo de Movimento com Validação:** Movimentação do sapo na grelha através do teclado, utilizando as setas direcionais ou as teclas WASD. O sistema valida cada movimento, impedindo a personagem de sair dos limites do mapa.
- **Movimento Autónomo de Entidades:** Atualização automática da posição dos predadores (raposas) e dos nenúfares ao longo do tempo, gerida pelo temporizador principal (*Game Loop*). Com lógica para a movimentação dos predadores implementada na classe.

- **Evolução Visual da Personagem:** Alteração do tamanho gráfico do sapo (fases Pequeno, Médio e Grande) de acordo com a sua progressão e estado no jogo.
- **Gestão de Vidas e Dano:** Controlo do número de vidas do jogador, com penalização (perda de uma vida) ao colidir com uma raposa, ao cair na água ou ao tentar sair do mapa a partir de um nenúfar. O estado atual é exibido graficamente no ecrã através de ícones de corações.
- **Recolha de Grilos e Pontuação:** Detecção de colisão para a recolha de grilos, que incrementam a pontuação do jogador. Implementação de itens especiais (*powerUps*) que adicionam uma vida extra ou fazem o sapo avançar três casas para a direita de forma imediata.
- **Progressão de Níveis:** Divisão do jogo em três níveis de dificuldade progressiva. A passagem de nível ocorre ao atingir a placa "Prox Nvl" nos dois primeiros cenários, e a vitória final é alcançada ao interagir com a Princesa no terceiro nível.
- **Menu de Pausa:** Interrupção do jogo a qualquer momento através da tecla ESC. Esta funcionalidade congela o temporizador e as entidades, exibindo opções para retomar a partida ou regressar ao menu principal.
- **Efeitos Visuais nos Componentes:** Implementação de reatividade nas interações em todos os botões da interface, alterando a sua escala quando o rato passa por cima ou quando são clicados.
- **Persistência de Dados (Leaderboard):** Gravação e leitura automática das dez melhores pontuações num ficheiro de texto local (leaderboard.txt), garantindo a atualização permanente da tabela do top 10.

As figuras seguintes (2-11) demonstram a interface gráfica e possíveis interações da aplicação.

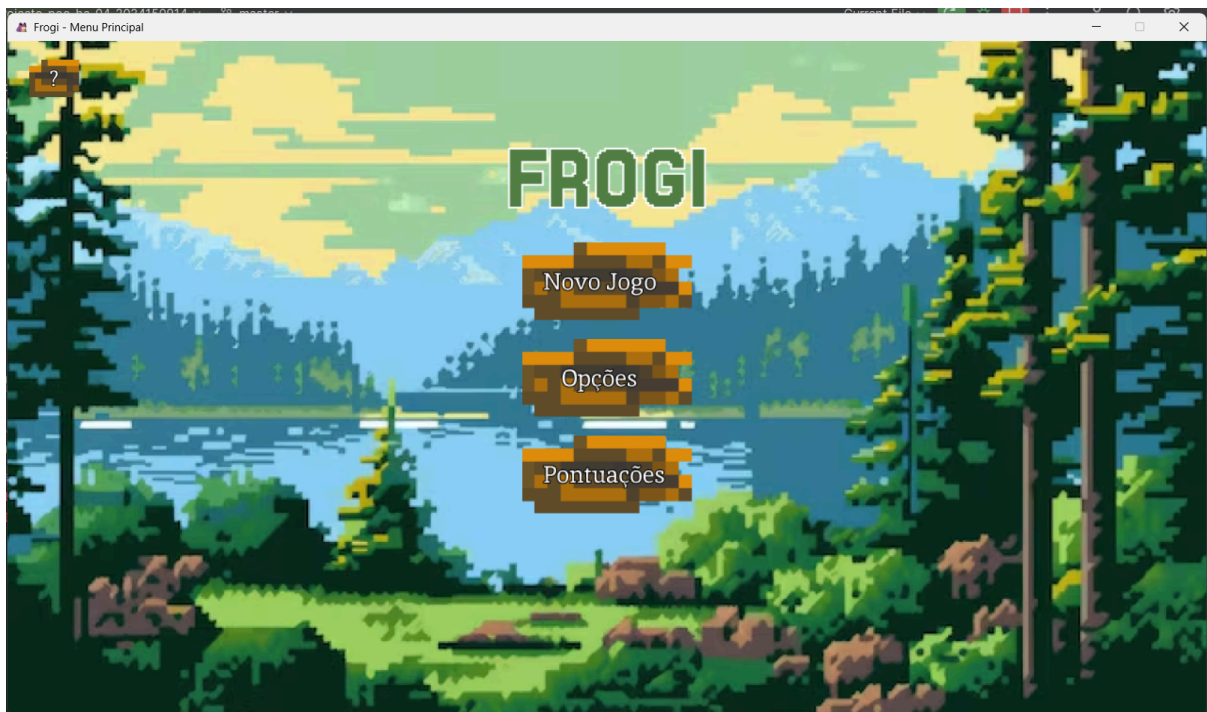


Figura 2 - Página Menu da Aplicação

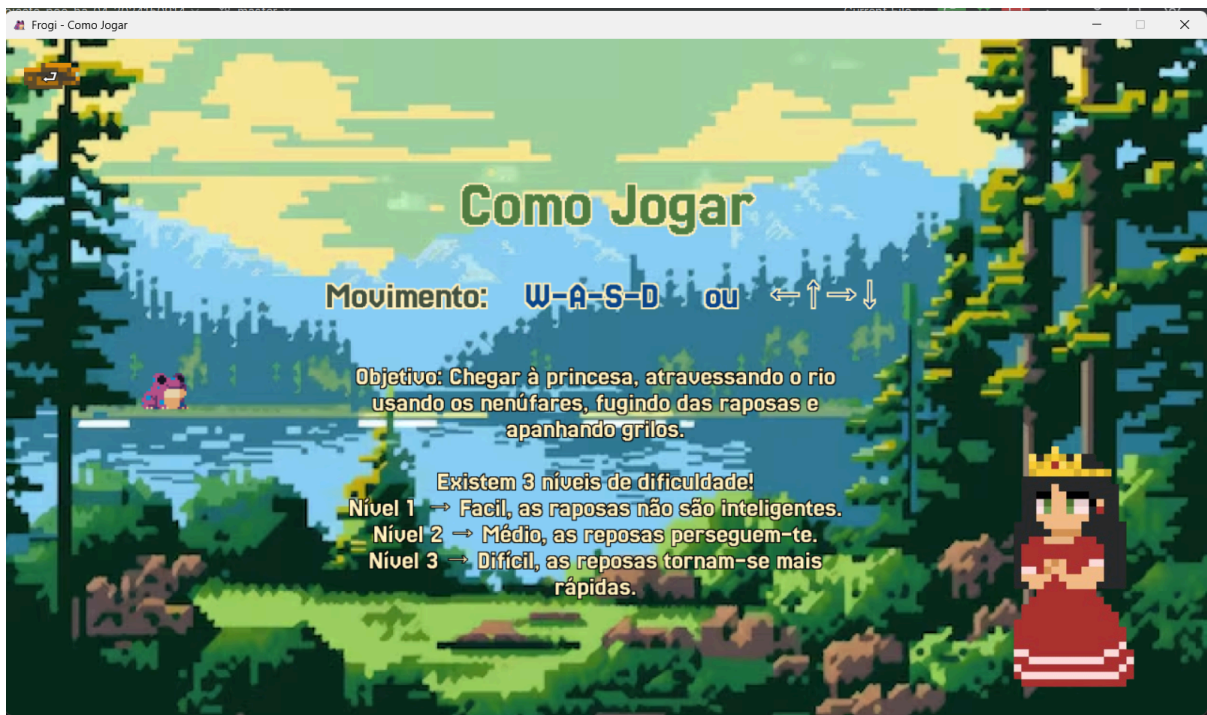


Figura 3 - Página Como Jogar da Aplicação

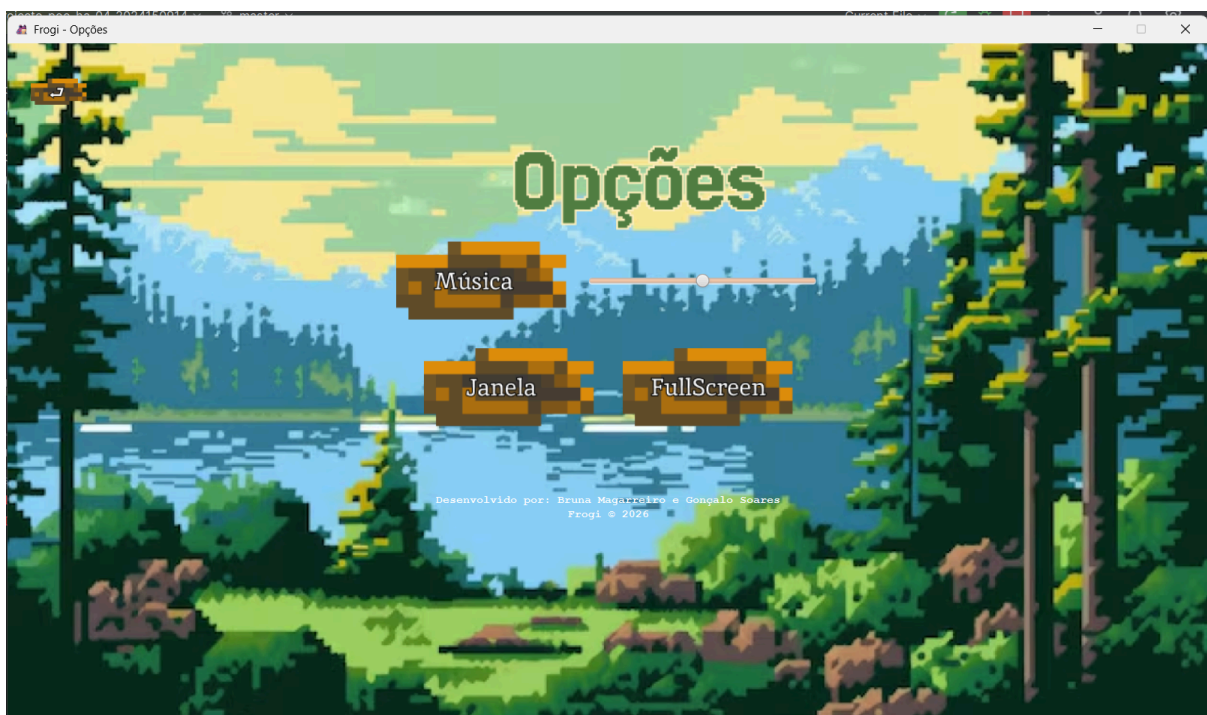


Figura 4 - Página Opções da Aplicação

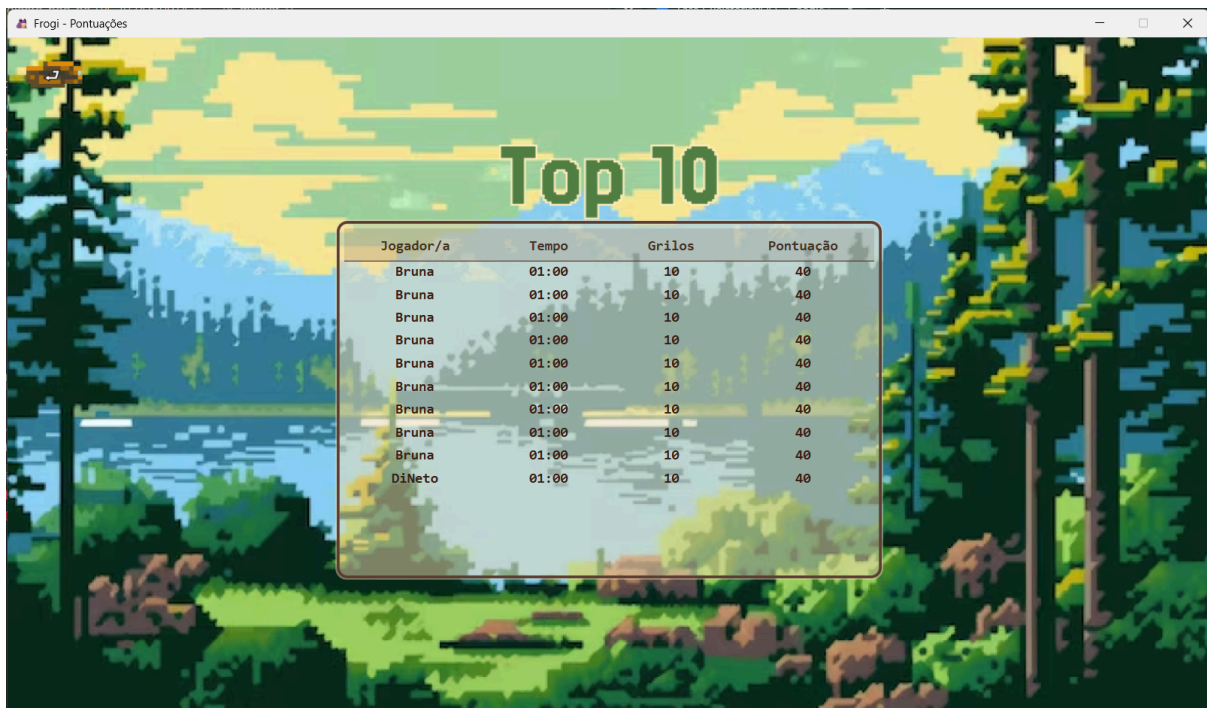


Figura 5 - Página Pontuações da Aplicação

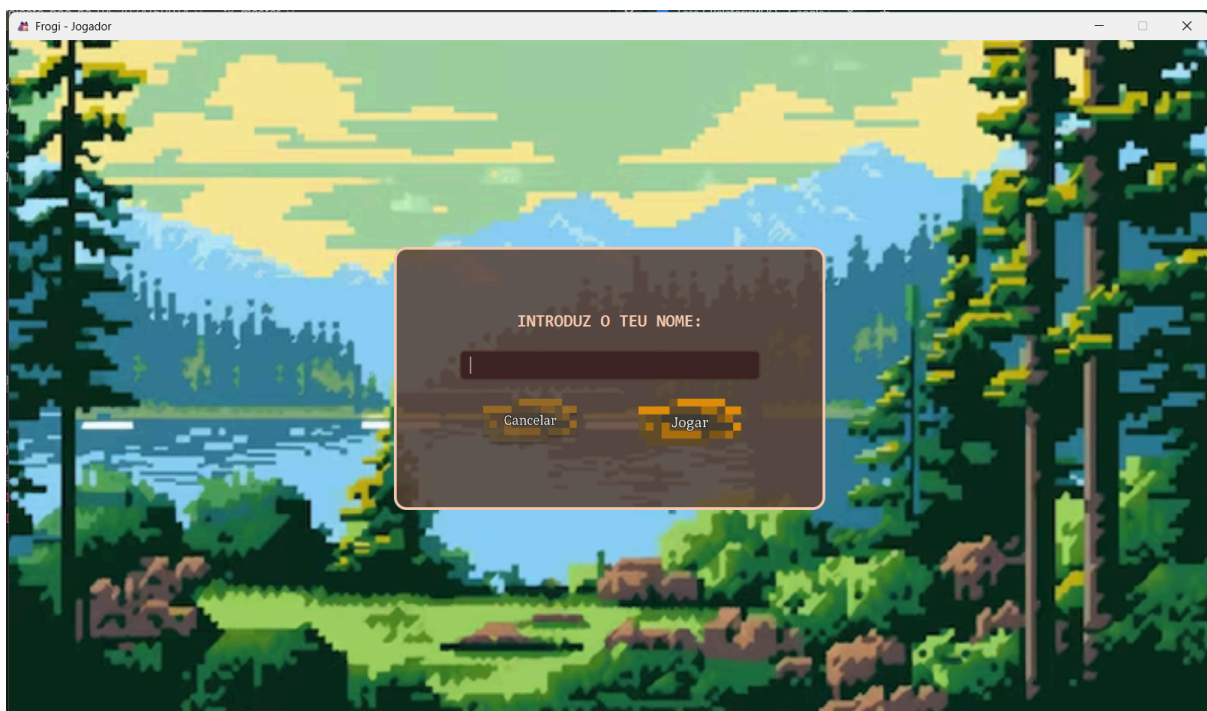


Figura 6 - Página Introduzir Nome da Aplicação



Figura 7 - Página Jogo da Aplicação - Nível 1

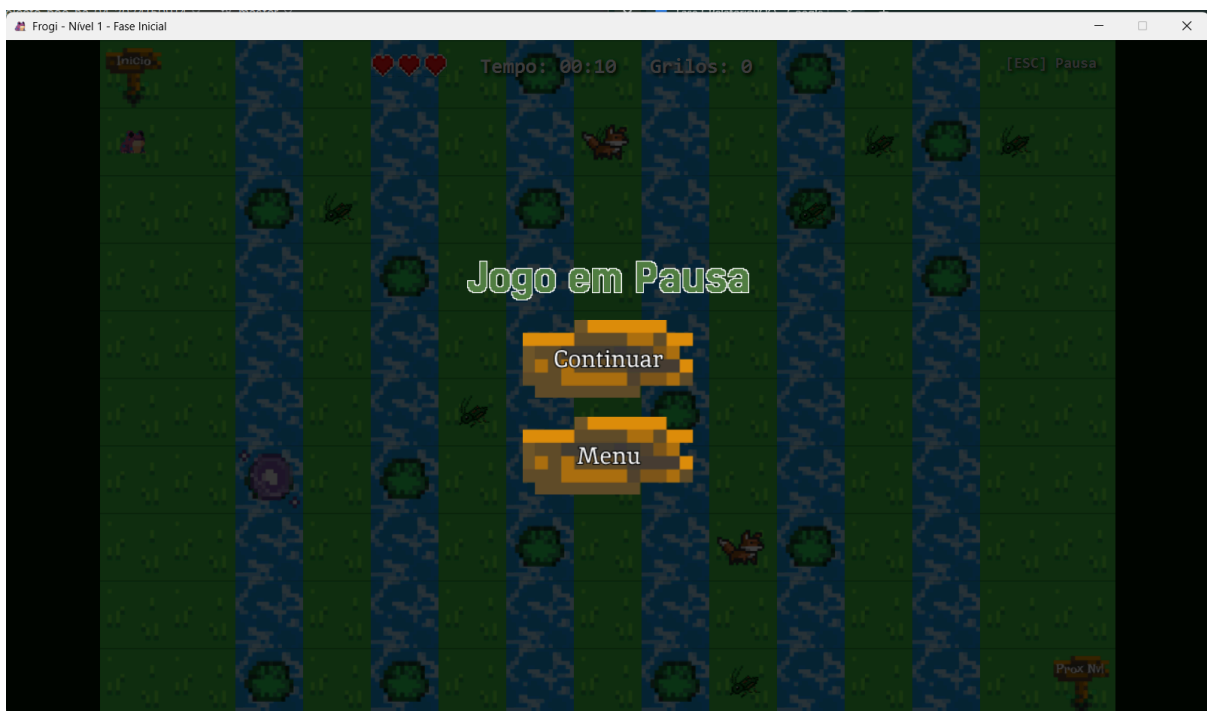


Figura 8 - Ecrã de Pausa do Jogo da Aplicação

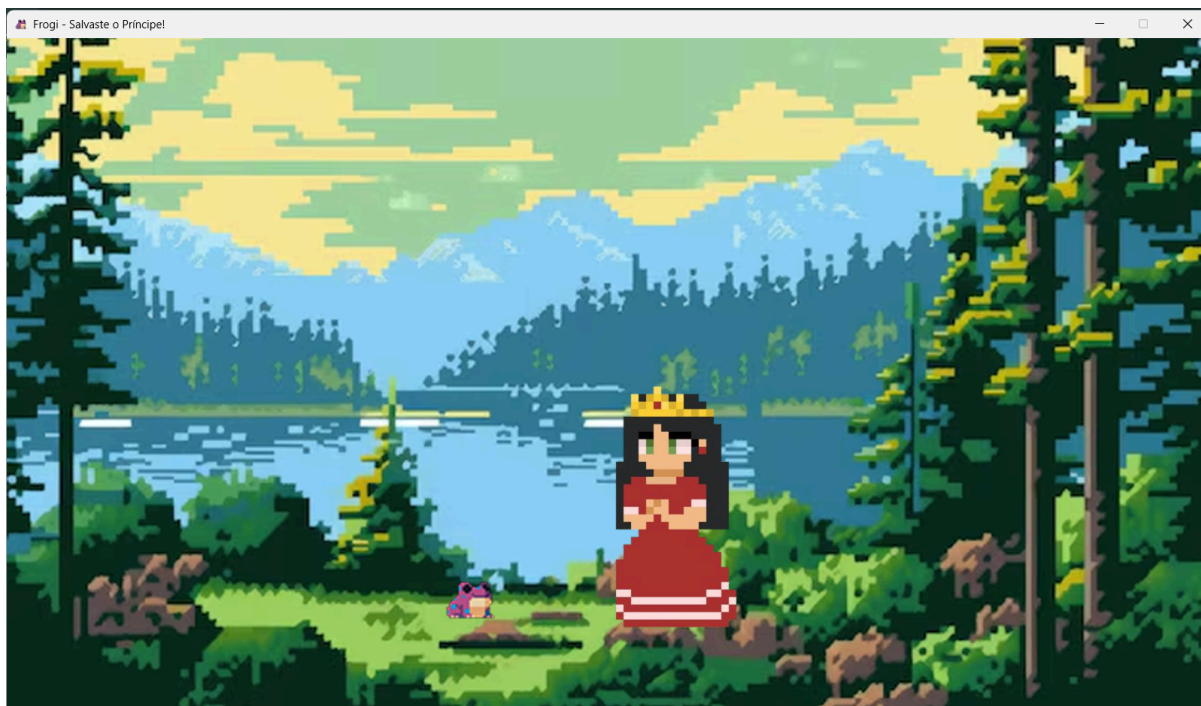


Figura 9 - Ecrã Inicial de Vitória

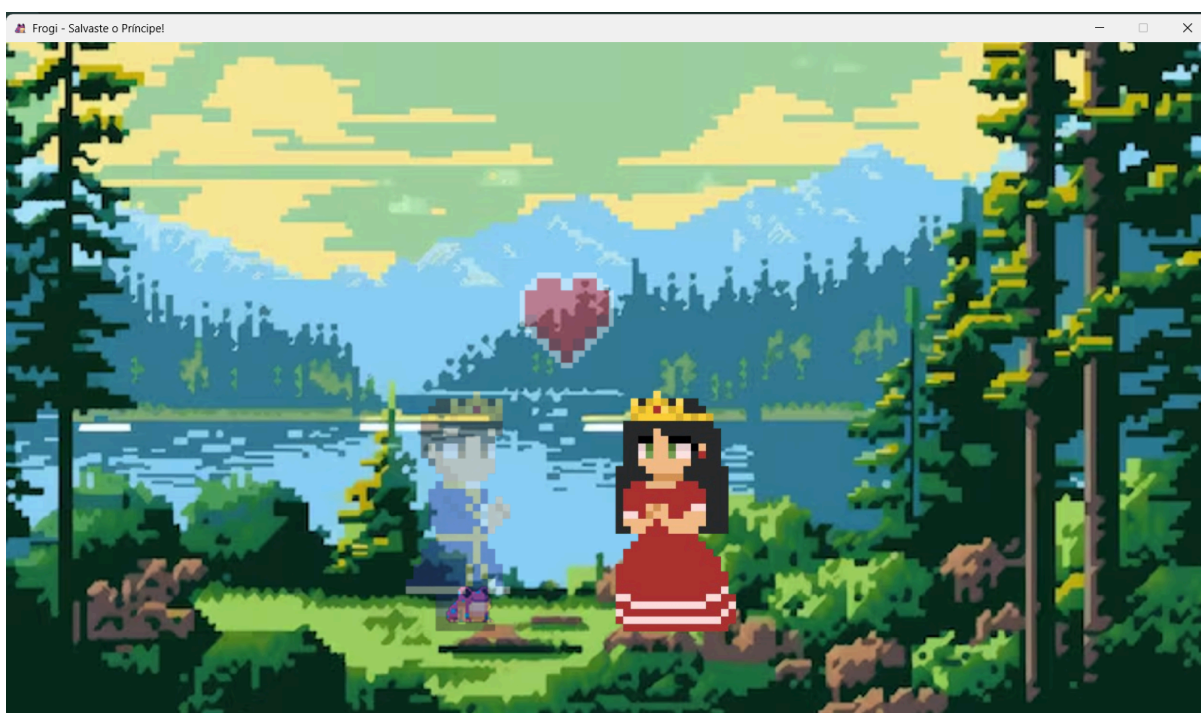


Figura 10 - Transição entre Ecrãs de Vitória

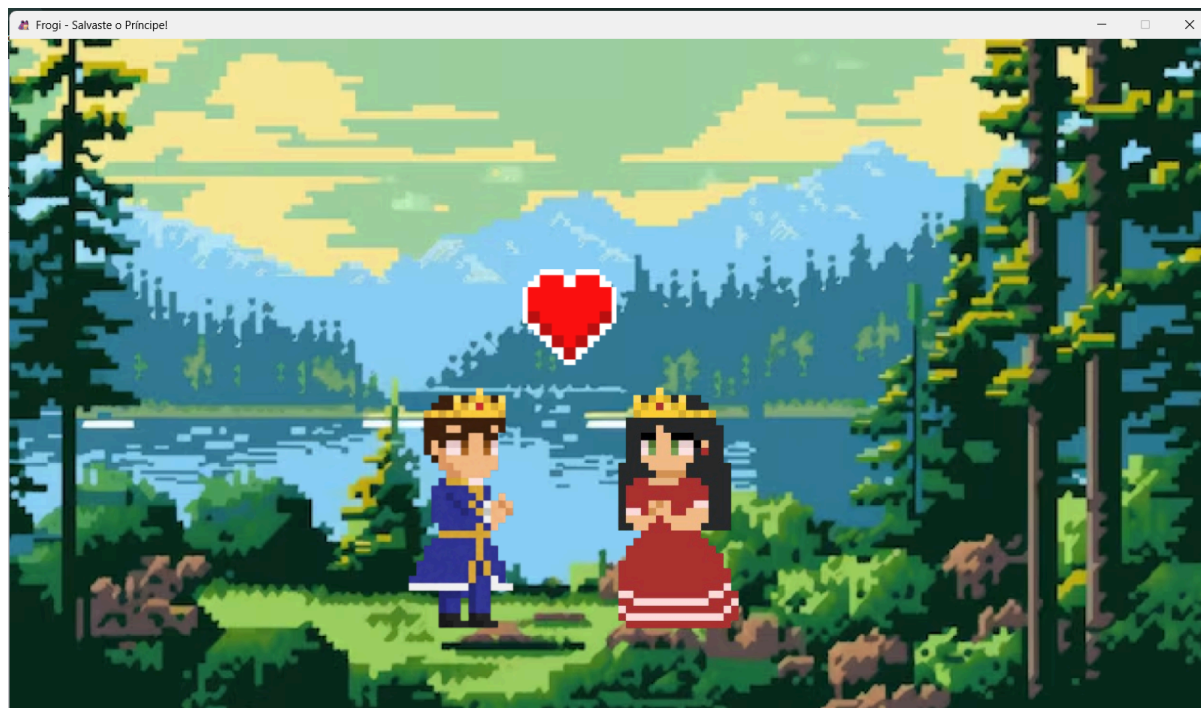


Figura 11 - Ecrã de Vitória Final

6. Aplicação dos Conceitos de POO

A estrutura e o desenvolvimento do código foram baseados nos pilares fundamentais da Programação Orientada a Objetos, garantindo a organização e a flexibilidade do sistema:

- **Encapsulamento:** É o mecanismo que agrupa os dados e os métodos que operam sobre esses dados, protegendo-os do acesso externo direto.

Para proteger o estado interno dos objetos contra modificações indevidas, todos os atributos das classes (como as coordenadas `posicaoX` e `posicaoY` das entidades, os dados do Mapa e as vidas na classe `Partida`) foram declarados como privados (`private`). O acesso e a alteração destes valores são feitos exclusivamente através de métodos públicos (`getters` e `setters`), que validam as operações, por exemplo, impedindo que o sapo se mova para fora dos limites definidos na grelha do mapa. Adicionalmente, aplicou-se o princípio da cópia defensiva nos métodos que expõem coleções de dados, como as listas de entidades ou colunas do rio. Em vez de devolver a referência direta do objeto interno, o método cria e envia uma nova cópia da coleção. Isto impede que classes externas modifiquem diretamente o conteúdo do mapa sem passar pelo controlo e validação do modelo.

- **Herança:** Permite criar uma nova classe a partir de uma classe já existente, herdando os seus atributos e métodos e promovendo a reutilização de código.

Utilizou-se a herança para reaproveitar código e agrupar comportamentos comuns. Criou-se a classe abstrata `EntidadeJogo`, que serve de superclasse para todos os elementos que existem no tabuleiro, partilhando os atributos de posicionamento. A partir dela, derivam as subclasses específicas como `Predador`, `Grilo`, `Princesa` e a classe abstrata `PowerUp` (que, por sua vez, é estendida por `VidaExtra` e `Salto`). Isto evitou a repetição de atributos idênticos em múltiplas classes.

- **Polimorfismo:** É a propriedade que permite tratar objetos de diferentes subclasses de forma genérica através da sua superclasse comum, invocando comportamentos específicos em tempo de execução.

Este conceito foi aplicado principalmente na gestão e na renderização do mapa. A classe Mapa armazena todas as personagens e objetos numa lista genérica do tipo List<EntidadeJogo>. Quando o jogo precisa de atualizar o ecrã ou verificar colisões, percorre esta lista e trata cada elemento de forma polimórfica. Em tempo de execução, o sistema sabe exatamente se deve aplicar o comportamento de movimento de uma raposa ou o comportamento de recolha de um grilo, sem a necessidade de criar estruturas condicionais (if/else) complexas e acopladas.

7. Tratamento de Exceções

Para garantir o correto funcionamento da aplicação e evitar falhas críticas em tempo de execução, foi desenvolvido um plano de tratamento de erros. Este plano divide-se na utilização de exceções nativas do Java e na criação de exceções personalizadas para regras específicas do negócio, organizadas no diretório org.frogi.model.exceptions.

Exceções Personalizadas:

- **NomeInvalidoException** (Herda de IllegalArgumentException): Lançada durante o registo do utilizador no ecrã de "Novo Jogo". Esta exceção é ativada se o jogador tentar iniciar a partida com o campo do nome vazio, com espaços em branco ou com caracteres não permitidos, impedindo o avanço do fluxo até que um nome válido seja inserido.

Escolheu-se herdar de IllegalArgumentException porque o nome introduzido pelo utilizador funciona como um argumento de entrada para iniciar o jogo. Como a classe IllegalArgumentException serve nativamente no Java para sinalizar que um método recebeu um argumento inválido, esta herança respeita as boas práticas da linguagem.

- **PosicaoInvalidaException** (Herda de RuntimeException): Utilizada pelo modelo de domínio para validar a movimentação das entidades na grelha. É lançada sempre que o sistema deteta uma tentativa de colocar ou mover o sapo para fora dos limites estabelecidos para o mapa, ou numa coordenada logicamente inacessível.

Optou-se por herdar de RuntimeException porque uma coordenada fora dos limites representa um erro de lógica interna do programa e não uma falha externa recuperável. Sendo uma exceção não verificada (unchecked), o erro é propagado de forma limpa até ao controlador sem sobrecarregar o código do modelo com blocos try-catch desnecessários.

- **EstadoSapoInvalidoException** (Herda de IllegalStateException): Garante a integridade do estado da personagem principal. É lançada se ocorrer uma tentativa de movimentar ou alterar o estado do sapo quando este já se encontra no estado de "morto".

Optou-se por herdar de IllegalStateException porque esta exceção nativa do Java indica que um método foi invocado num momento em que o ambiente ou o objeto não se encontram num estado apropriado para essa operação. Neste caso, tentar mover um sapo que logicamente já não tem vidas é uma violação do estado correto do fluxo do jogo.

Exceções Nativas do Java:

- **IOException:** Tratada no mecanismo de persistência de dados. Sempre que o jogo realiza a leitura ou a escrita no ficheiro local leaderboard.txt, a operação é envolvida num bloco try-catch. Caso ocorra uma falha de permissões no disco ou o ficheiro não exista, a exceção é capturada, evitando o colapso do programa e garantindo que o ecrã de pontuações exiba uma tabela limpa.
- **NullPointerException e IllegalArgumentException:** Tratadas no carregamento dos recursos visuais (imagens) e sonoros através das classes de controlo. Se algum ficheiro .png ou de áudio estiver em falta dentro do arquivo, o erro é intercetado para que a aplicação utilize uma cor de substituição ou ignore o efeito sonoro, mantendo o jogo executável.

8. Testes Unitários

Para garantir a fiabilidade das regras de negócio do jogo, foram desenvolvidos testes unitários com a biblioteca JUnit (localizados na diretoria src/test/java/org.frogi.model). Os testes focam-se exclusivamente no modelo de domínio (org.frogi.model), validando o comportamento das entidades e a consistência das pontuações de forma isolada da interface gráfica.

Os principais métodos testados e os cenários considerados foram:

- **Validação de Movimento do Sapo:**
 - Métodos testados: Métodos de deslocação e atualização de posição na classe Sapo.
 - Cenários considerados: Verificação do movimento correto nas quatro direções (cima, baixo, esquerda, direita) numa grelha vazia e validação do bloqueio ou disparo da exceção PosicaoInvalidaException quando o sapo tenta mover-se para além dos limites do mapa.
- **Sistema de Colisões e Gestão de Vidas:**
 - Métodos testados: Métodos de deteção de interseções na classe Mapa.
 - Cenários considerados: Validação de colisão direta entre o Sapo e um Predador (raposa), confirmando a redução imediata de uma vida. Testou-se também o cenário de queda na água e a transição para o estado de derrota assim que o contador de vidas chega a zero.
- **Recolha de Itens e Ativação de Efeitos:**
 - Métodos testados: Métodos de interação com entidades estáticas e powerUps.
 - Cenários considerados: Validação do incremento da pontuação ao colidir com um Grilo. Teste do efeito do power-up de VidaExtra (confirmando o ganho de um coração) e do power-up de Salto (verificando se a posição X do sapo avança as 3 coordenadas estipuladas).
- **Progressão de Níveis e Vitória:**
 - Métodos testados: Métodos de atualização de estado da classe Partida.
 - Cenários considerados: Passagem automática para a fase seguinte ao colidir com a entidade da placa "Prox Nvl" nos Níveis 1 e 2, e validação do estado

de vitória da partida ao alcançar a coordenada ocupada pela Princesa no Nível 3.

- **Persistência e Gravação de Resultados:**
 - Métodos testados: Métodos de leitura, escrita e ordenação de pontuações na classe de gestão de resultados (ResultadoPartida).
 - Cenários considerados: Este bloco exigiu uma complexidade técnica superior, dado que lida diretamente com o sistema de ficheiros. Foram criados cenários para garantir que o sistema grava corretamente uma nova pontuação, ordena a lista por ordem decrescente e mantém apenas o Top 10 dos jogadores. Para evitar corromper o ficheiro real de recordes do jogo durante a execução dos testes, implementou-se uma lógica de isolamento que cria e limpa dados de teste num ficheiro temporário antes e depois de cada verificação.

9. Dificuldades Encontradas

Durante o desenvolvimento do projeto, surgiram alguns desafios técnicos que exigiram uma análise detalhada para encontrar as soluções mais adequadas:

- **Configuração do Launcher e Ciclo de Vida do JavaFX:** Inicialmente, verificou-se uma incompatibilidade ao tentar iniciar a aplicação diretamente a partir da classe principal que estende Application do JavaFX, resultando em erros de inicialização no ambiente de execução. Para contornar este problema, criou-se uma classe separada com um método main simples (um Launcher independente) que apenas faz a chamada para o arranque da aplicação JavaFX. Isto isolou o ciclo de vida da interface gráfica e permitiu que o programa iniciasse corretamente.
- **Gestão de Caminhos de Recursos:** O carregamento de ficheiros externos, como as imagens e os ficheiros de áudio, falhava frequentemente ao executar o projeto noutros computadores. Para resolver este problema substituiu-se a utilização de caminhos relativos e absolutos pelo método getClass().getResource(). Desta forma, todos os recursos passaram a ser tratados como fluxos de dados internos do próprio projeto, garantindo que a aplicação encontra os ficheiros independentemente de onde esteja a ser executada.
- **Sincronização do Game Loop com a Interface Gráfica:** Coordenar o movimento autónomo e contínuo das raposas e dos nenúfares com a taxa de atualização do ecrã foi um desafio. Inicialmente, atualizar a lógica do mapa diretamente a partir de loops secundários gerava conflitos visuais e bloqueios na interface gráfica do JavaFX. Utilizou-se a classe Timeline do JavaFX para gerir o Game Loop principal. Sendo executada na mesma thread gráfica da aplicação, a Timeline permitiu definir ciclos de atualização regulares onde o controlador avisa o modelo para mover os elementos e, de seguida, redesenha o ecrã de forma fluida e sincronizada.
- **Isolamento de Testes no Sistema de Ficheiros:** Conforme mencionado no ponto anterior, testar unitariamente a classe responsável por ler e escrever o Top 10 de jogadores revelou-se complexo. Escrever dados diretamente no ficheiro oficial leaderboard.txt durante os testes automáticos corrompia os resultados reais

guardados. A abordagem adotada passou por parametrizar o caminho do ficheiro na classe de persistência (ResultadoPartida). Nos testes unitários, o JUnit foi configurado para criar e apontar para um ficheiro temporário exclusivo para testes, que é limpo e apagado automaticamente após a execução de cada cenário, salvaguardando o ficheiro de pontuações real.

10. Conclusão

O desenvolvimento do jogo Frogi permitiu cumprir com sucesso todos os objetivos inicialmente propostos para o projeto. A aplicação final apresenta-se como um jogo funcional, dinâmico e interativo, reunindo todas as regras de negócio planeadas. Desde a movimentação validada do sapo e o comportamento autónomo dos predadores e nenúfares, até ao sistema de progressão por três níveis de dificuldade e a persistência local de resultados.

O projeto foi fundamental para consolidar conceitos práticos de engenharia de software e programação orientada a objetos. A divisão rigorosa da arquitetura em três camadas independentes (Model, View e Controller) demonstrou a importância de criar código com baixo acoplamento, o que facilitou diretamente a implementação de testes unitários isolados com o JUnit. Além disso, o desenvolvimento da interface gráfica e do sistema sonoro com o JavaFX permitiu compreender na prática a gestão de eventos, o ciclo de vida de aplicações visuais e a importância do tratamento de exceções para garantir a estabilidade do programa perante erros de ficheiros ou de lógica.

Em suma, o projeto representou um desafio enriquecedor que traduziu conceitos teóricos em soluções de código limpas, seguras e organizadas, resultando numa aplicação estável, escalável e totalmente operacional.